# *Lab 2 Report: ROS, TF and Quaternions*

*Xuanye Xu*

Student ID 24020036071 / H00490299

Robotics Integration I — VNAV Labs

*September 25, 2026*

ROS 1 Noetic was installed in Ubuntu 20.04 under WSL 2 and verified with turtlesim. In the two-drone scenario, AV1 flies a circle and AV2 an arc of a parabola; their poses were published as `tf2` transforms, and the relative transform was looked up to plot three trajectories in RViz. Seen from AV1, AV2 moves on an ellipse in the plane $z - 2y = 1$ with semi-axes $\frac{1}{2}$ and $\frac{\sqrt{5}}{2}$; this was derived with homogeneous transforms and confirmed on 300 tf samples to round-off level. Three properties of the quaternion multiplication matrices were proved.

## *Grading Map*

| Grading item | Points | Section | Evidence |
|---|---|---|---|
| ROS installation | 20 | Section 2 | Environment table, turtlesim screenshot, node and topic listing |
| Two-drone motion | 40 | Section 3 | Answers on nodes, topics and the launch file; publisher code; static and dynamic scenes; TF tree; `/tf` rate |
| Trajectories | 30 | Section 4 | Three trajectories in the `world` and `av1` frames; numerical check against the analytic model |
| Mathematical derivation | 10 | Section 5 | Five derivations with homogeneous transforms |
| Quaternion properties | 10 | Section 6 | Three proofs; note on a sign difference in the lecture slides |

## 1 *Lab Introduction*

This lab introduces the Robot Operating System (ROS 1 Noetic) and the `tf2` transform library. Two simulated drones are driven by publishing their poses as time-varying transforms, their trajectories are drawn in RViz in different reference frames, and the observed relative motion is explained with homogeneous transforms. The last part proves several properties of the quaternion multiplication matrices. The tasks follow the MIT VNAV 2023 Lab 2 exercises,[1] using the ROS 1 template code from the `VNAV2023` release.[2] The working environment is summarised in Table 1.

[1] MIT SPARK Lab. VNAV 2023 Lab 2: ROS Exercises. https://vnav.mit.edu/labs_2023/lab2/exercises.html, 2023a

[2] MIT SPARK Lab. VNAV-labs, release VNAV2023 (commit 7d234d8). https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2023, 2023b

## 2 *ROS Installation* [20 pts]

*Platform.* All ROS work runs in Ubuntu 20.04 under WSL 2 on an x86-64 Windows 11 desktop (Table 1). Commands are issued from a MacBook over SSH; the Windows desktop is viewed with Moonlight, and Linux GUI programs (turtlesim, RViz) are displayed on it through WSLg.

Apple-silicon Macs cannot run the x86-64 simulators required in Labs 3 and 4, so the whole course uses this x86-64 host.

| Component | Version |
|---|---|
| Windows host | 10.0.26200 (Windows 11), WSL 2.7.12, WSLg 1.0.73 |
| Linux | Ubuntu 20.04.6 LTS, kernel 6.18.33.2-microsoft-standard-WSL2, x86_64 |
| ROS | Noetic (`ros-noetic-desktop-full` 1.5.0) |
| Key packages | roscpp 1.17.4, tf2_ros 0.7.10, rviz 1.14.26, turtlesim 0.10.3 |
| Build tools | catkin_tools 0.9.4, g++ 9.4.0, CMake 3.16.3 |
| Lab code | VNAV-labs tag `VNAV2023`, commit `7d234d8` |

Table 1: Software environment, taken from `results/logs/environment.txt`.

*Installation.* The ROS key used by older installation guides expired on 1 June 2025, so the package source was configured with the official `ros-apt-source` package (version 1.3.0), which installs the current signing key and the `packages.ros.org` source.[3] The desktop-full metapackage and the build tools were then installed with `apt`: 1487 new packages, including 234 `ros-noetic-*` packages, with no apt errors or warnings. Finally `rosdep` was initialised and updated with `-include-eol-distros`, because Noetic reached end of life in May 2025, and `source /opt/ros/noetic/setup.bash` was added to `~/.bashrc`.

[3] Open Robotics. ROS signing key migration guide. https://discourse.openrobotics.org/t/ros-signing-key-migration-guide/43937, 2025

Full command list and apt summary: `results/logs/ros_install_summary.txt`.

*Turtlesim test.* `roscore`, `turtlesim_node` and `turtle_teleop_key` were started in three shells. The teleoperation node reads the arrow keys and publishes `geometry_msgs/Twist` messages on `/turtle1/cmd_vel`, which the simulator subscribes to (Listing 1). While the turtle was driven by hand, all 32 published commands were recorded: 12 forward and 2 backward ($v_x = \pm 2$), 7 left and 11 right turns ($\omega_z = \pm 2$). Figure 1 shows the resulting path.

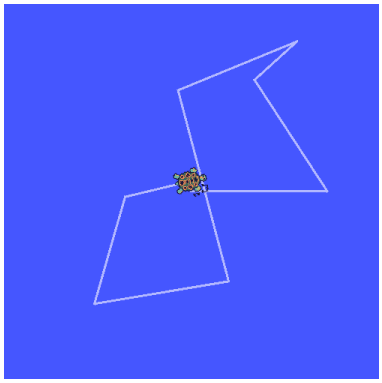


Figure 1: Turtlesim window after keyboard teleoperation; the trail is the path driven with the arrow keys.

Listing 1: Nodes and the command topic during teleoperation (`results/logs/turtlesim_nodes_topics.txt`).

```
--- rosnode list
/rosout
/teleop_turtle
/turtlesim
--- rostopic info /turtle1/cmd_vel
Type: geometry_msgs/Twist

Publishers:
 * /teleop_turtle (http://DESKTOP-91LK43L:44136/)

Subscribers:
 * /turtlesim (http://DESKTOP-91LK43L:44040/)
```

> EVIDENCE Table 1 (versions); `ros_install_summary.txt` (installation); Figure 1 (turtlesim with keyboard-driven path); Listing 1 (nodes and topic connections); `turtle_cmd_vel.csv` (32 recorded keyboard commands).

## 3 *ROS Basics and Two-Drone Motion* [40 pts]

The package `two_drones_pkg` was taken unchanged from the `VNAV2023` release[4] and built with `catkin build` in `~/vnav_ws`. Before the first build one defect of the template was fixed (Listing 2):

[4] MIT SPARK Lab. VNAV-labs, release VNAV2023 (commit 7d234d8). https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2023, 2023b

`plots_publisher_node` declared its `tf_listener` before the `tf_buffer` it is constructed from. C++ constructs members in declaration order, so the listener received a reference to a buffer that did not exist yet; reordering the declarations removes this undefined behaviour. The loop that waits for the first transforms also received a 0.1 s sleep so that it no longer spins at full CPU. All builds finished without compiler warnings.

Listing 2: Member declarations in `plots_publisher_node.cpp` after the fix.

```
// tf_buffer must be declared before tf_listener: members are
   constructed in
// declaration order, and tf_listener is constructed from tf_buffer.
tf2_ros::Buffer tf_buffer;
tf2_ros::TransformListener tf_listener;
```

### 3.1 *Nodes, topics and the launch file*

The static scenario was started with `roslaunch two_drones_pkg two_drones.launch static:=True` (Figure 3); all answers below were checked on the running system.

Raw output: `results/logs/static_nodes_topics.txt`, `manual_static_run.txt`, `nonstatic_template_run.txt`.

*Q1: running nodes.* Apart from `/rosout`, which belongs to `roscore`, four nodes run: `/av1broadcaster` and `/av2broadcaster` (both `tf2_ros/static_transform_publisher`), `/plots_publisher_node` and `/rviz`.

*Q2: the same scenario without roslaunch.* Each command of Listing 3 runs in its own terminal after sourcing the workspace. The `__name:=` remappings give the nodes the names used in the launch file; without the last one RViz registers under an anonymous name such as `/rviz_1790321167892688005`. With these commands the node list and the `/tf_static` connections were identical to the launch-file run, and RViz showed the same scene.

Listing 3: Commands that reproduce the static scenario, one per terminal.

```
roscore
rosrun tf2_ros static_transform_publisher 1 0 0 0 0 0 1 world av1 __name:=
     av1broadcaster
rosrun tf2_ros static_transform_publisher 0 0 1 0 0 0 1 world av2 __name:=
     av2broadcaster
rosrun two_drones_pkg plots_publisher_node
rosrun rviz rviz -d $(rospack find two_drones_pkg)/config/default.rviz __name:=rviz
```

*Q3: topics.* Table 2 lists the connections reported by `rosnode info`. The frames `av1` and `av2` are published by `/av1broadcaster` and `/av2broadcaster` on `/tf_static`. The drone meshes come from the `visualization_msgs/MarkerArray` on `/visuals`, published by `/plots_publisher_node`: unticking the MarkerArray display removes both meshes while the TF axes stay (Figure 2).

| Node | Publishes | Subscribes |
|---|---|---|
| `/av1broadcaster` | `/tf_static` | — |
| `/av2broadcaster` | `/tf_static` | — |
| `/plots_publisher_node` | `/visuals` | `/tf`, `/tf_static` |
| `/rviz` | `/clicked_point`, `/initialpose`, `/move_base_simple/goal` | `/tf`, `/tf_static`, `/visuals` |

Table 2: Topics of the static scenario, `/rosout` omitted. The three topics published by RViz belong to its interactive tools and have no subscribers here.

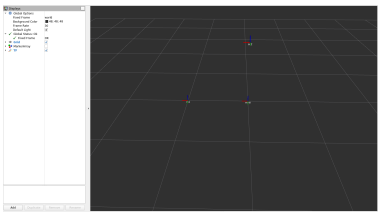


Figure 2: Static scene with the MarkerArray display unticked: the meshes disappear, the TF axes of `world`, `av1` and `av2` remain.

*Q4: omitting `static:=True`.* The launch file declares `<arg name="static" default="false"/>`. The two static publishers sit in `<group if="$(arg static)">` and are started only when the argument is true, while `frames_publisher_node` carries `unless="$(arg static)"` and is started only when it is false. Without the argument the launch therefore replaces the fixed transforms by the time-varying ones of `frames_publisher_node`. With the unmodified template this node publishes nothing, which was observed directly (Listing 4): `/tf` has no publisher, RViz reports "No TF data" for the fixed frame, and `plots_publisher_node` stays in its wait loop, so `/visuals` carries no messages. After Deliverable 2 the same launch shows the moving drones.

Listing 4: Launch without `static:=True` before the publisher was implemented; exit code 124 means `timeout` expired before any `/tf` message arrived.

```
--- rosnode list
/frames_publisher_node
/plots_publisher_node
/rosout
/rviz
--- rostopic info /tf
Type: tf2_msgs/TFMessage

Publishers: None
--- rostopic echo -n 1 /tf (5 s timeout)
exit=124 lines=0
--- rostopic hz /visuals (5 s)
no new messages
```

## 3.2 *Publishing the drone frames*

Listing 5 shows the code added to `onPublish()`, which the timer calls every 20 ms. The clock is read once per call: the elapsed time $t$ in seconds drives both poses, and the same instant is used as the stamp of both transforms, so a lookup between `av1` and `av2` combines poses of the same moment. AV1 is placed at $(\cos t, \sin t, 0)$ with roll $=$ pitch $=$ 0 and yaw $= t$. Its $y$ axis is then $R_z(t)\,\boldsymbol{e}_y = (-\sin t, \cos t, 0)^T = \dot{\boldsymbol{o}}_1^w(t)$, the direction of motion, and $z_1 = z_w$, as required. A yaw of $t + \pi/2$ would point $x_1$ forward instead, which is the usual convention but not the one of this lab. AV2 is placed at $(\sin t, 0, \cos 2t)$ and keeps the identity rotation.

Listing 5: Transform computation in `frames_publisher_node.cpp`; the broadcaster is a class member.

```
// Read the clock once so that the elapsed time and both stamps agree.
const ros::Time now = ros::Time::now();
const double time = (now - startup_time).toSec();
AV1World.header.stamp = now;
AV1World.header.frame_id = "world";
AV1World.child_frame_id = "av1";
AV1World.transform.translation.x = std::cos(time);
AV1World.transform.translation.y = std::sin(time);
AV1World.transform.translation.z = 0.0;
// yaw = time turns y_1 into (-sin(time), cos(time), 0), the direction of motion
// along the circle; roll = pitch = 0 keeps z_1 parallel to z_w.
tf2::Quaternion q_av1;
q_av1.setRPY(0.0, 0.0, time);
AV1World.transform.rotation.x = q_av1.x();
AV1World.transform.rotation.y = q_av1.y();
AV1World.transform.rotation.z = q_av1.z();
AV1World.transform.rotation.w = q_av1.w();

// AV2 only translates, so it keeps the identity rotation set above.
AV2World.header.stamp = now;
AV2World.header.frame_id = "world";
AV2World.child_frame_id = "av2";
AV2World.transform.translation.x = std::sin(time);
AV2World.transform.translation.y = 0.0;
AV2World.transform.translation.z = std::cos(2 * time);
broadcaster.sendTransform(AV1World);
broadcaster.sendTransform(AV2World);
```

## 3.3 *Results*

Figures 3 and 4 show the static and the moving scene. During the dynamic run `/frames_publisher_node` was the only publisher on `/tf`; the topic carried 100 Hz of messages because each call sends two transforms, and in 5 s both `av1` and `av2` appeared 239 times. `view_frames` reports 50.2 Hz for each frame (Figure 5). The orientation requirement is confirmed numerically in Section 4.3: the yaw of `av1` equals the polar angle of its position to $4 \times 10^{-16}$ rad, and its roll/pitch part is exactly zero.

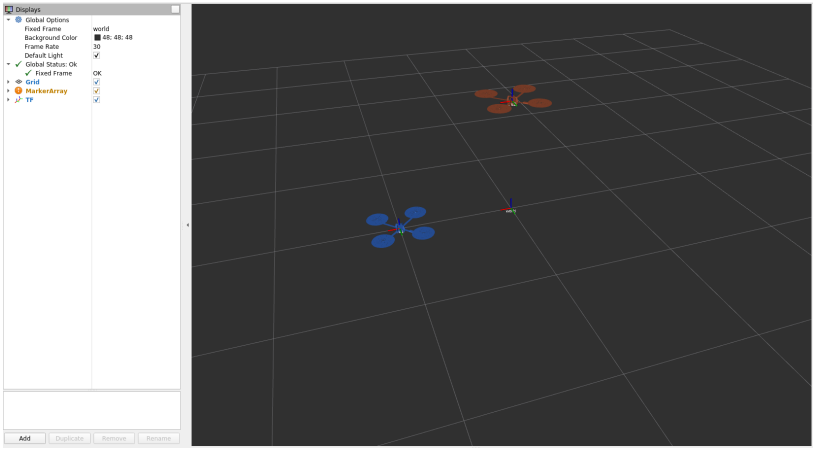



Figure 3: Static scenario, fixed frame `world`: AV1 (blue) at $(1, 0, 0)$, AV2 (red) at $(0, 0, 1)$, with the TF axes of all frames.

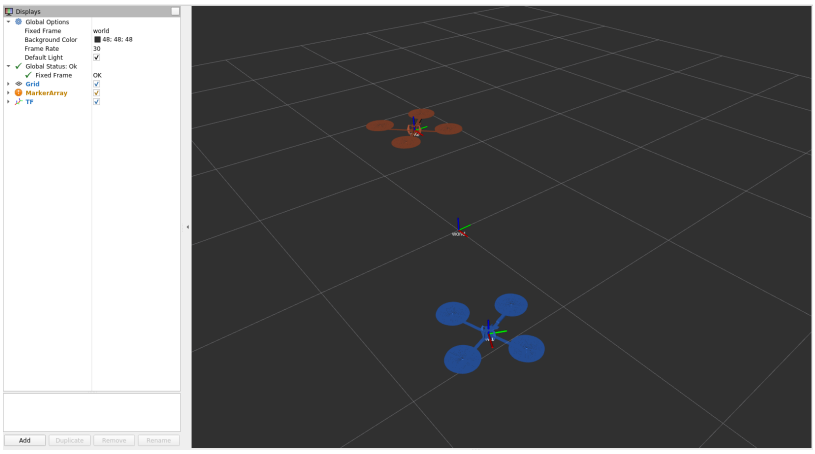


Figure 4: Moving scenario after Deliverable 2, before the trails were implemented. The green $y$ axis of AV1 points along the circle.

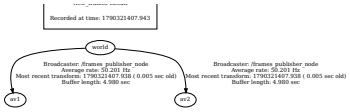



Figure 5: TF tree of the moving scenario (`rosrun tf view_frames`).

> EVIDENCE Table 2 and Listings 3 and 4 and Figure 2 (four questions); Listing 5 (implementation); Figures 3 and 4 (scenes); Figure 5 and `dynamic_nodes_topics.txt` (publisher and rates); Table 3 (orientation).

## 4 *Trajectories and Reference Frames* [30 pts]

### 4.1 *Querying the relative transform*

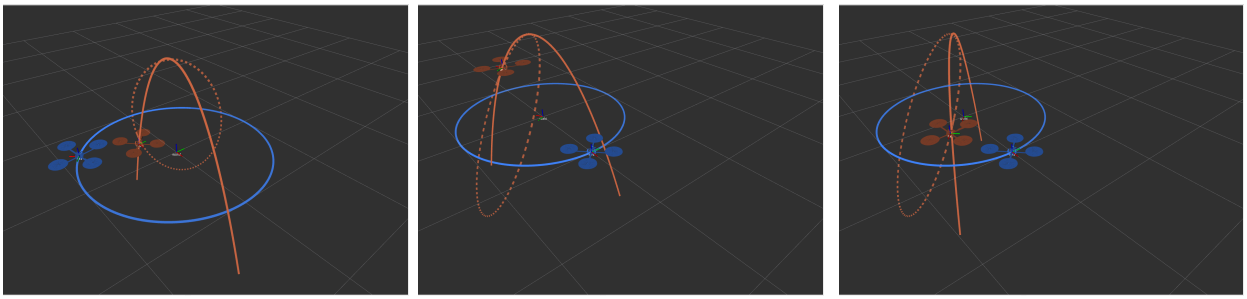
Each trail asks tf for the pose of `dest_frame` in `ref_frame` (Listing 6). `lookupTransform(target, source, time)` returns the transform that maps coordinates from the source frame into the target frame, i.e. $T^{\text{ref}}_{\text{dest}}$; its translation is the origin of `dest_frame` expressed in `ref_frame`, which is exactly the point to append to the trail. For the dashed trail the call is `lookupTransform("av1", "av2", ...)`, giving $\boldsymbol{o}_2^1$. Swapping the arguments would return $T_1^2$, whose translation $-R_z(t)\,\boldsymbol{o}_2^1$ traces a different curve. `ros::Time(0)` requests the latest transform in the buffer; a failed lookup raises `tf2::TransformException`, which the template catches, so the sample is skipped instead of adding a zero point. No lookup failed during the runs. Two small changes support the display: the trail markers now carry an identity orientation (RViz had warned about an uninitialised quaternion), and the world trails keep 400 samples (8 s at 50 Hz) instead of 300, so that one full $2\pi$ s period is visible.

Listing 6: Transform lookup in `TrajTrail::update()`.

```
53      // Pose of dest_frame expressed in ref_frame, i.e. T^{ref}_{
        dest};
54      // ros::Time(0) asks for the latest transform available in the
        buffer.
55      transform = parent->tf_buffer.lookupTransform(ref_frame,
        dest_frame, ros::Time(0));
```

### 4.2 *Trajectories in RViz*

Figure 6 shows the three trails. With fixed frame `world`, AV1 draws the unit circle (blue), AV2 the parabola arc in the $x$–$z$ plane (red, solid), and the dashed red curve is AV2's path in `av1`, drawn in that moving frame. With fixed frame `av1` the scene is seen from AV1: AV1 stays still, the world axes spin about their $z$ axis, and the parabola rotates with them (Figure 6 (b) and (c)). The world origin stays at $(-1, 0, 0)$ in `av1` because $R_z(t)^T(-\boldsymbol{o}_1^w) = (-1, 0, 0)^T$, and a circle turning about its own centre looks unchanged, which is why the blue trail appears fixed as well. The dashed trail is a closed curve on a slanted plane, the ellipse derived in Section 5.



(a) fixed frame `world` (b) fixed frame `av1` (c) fixed frame `av1`, a few seconds later

Figure 6: Trails in RViz. Blue solid: AV1 in `world`; red solid: AV2 in `world`; red dashed: AV2 in `av1`. From (b) to (c) AV1 stays in place while the world-frame parabola rotates. 3D view cropped; the full RViz windows, including the Fixed Frame setting, are in `results/screenshots/traj_fixed_*.png`.

### 4.3 *Numerical check*

To compare the tf data with the model, `scripts/check_tf.py` looked up `world`→`av1`, `world`→`av2` and `av1`→`av2` at a common stamp, 300 times over 12 s. The phase $t \bmod 2\pi = \operatorname{atan2}(y_1, x_1)$ of AV1 serves as the time argument of the closed-form expressions of Section 5. It grows at 1.000 000 rad/s against the message stamps, so $t$ in the publisher is indeed elapsed seconds. Table 3 lists the largest residuals; all are at double-precision round-off, and the plane coordinates of the samples fill the ellipse of Figure 7. This checks the geometry of the published and composed transforms; it does not measure timing, which is covered by the rates in Section 3.

| Quantity | Max. $\lvert\cdot\rvert$ |
|---|---|
| AV1: $x^2 + y^2 - 1$ | $2.2 \times 10^{-16}$ |
| AV1: yaw $-$ atan2$(y, x)$ [rad] | $4.4 \times 10^{-16}$ |
| AV1: $z$; roll/pitch part $\lVert(q_x, q_y)\rVert$ | 0 |
| AV2: $z - (1 - 2x^2)$ | $3.3 \times 10^{-16}$ |
| AV2: $y$; rotation angle | 0 |
| `av1`→`av2` vs. $\boldsymbol{o}_2^1(t)$ of eq. (1) [m] | $6.1 \times 10^{-16}$ |
| Plane: $z - 2y - 1$ | $6.7 \times 10^{-16}$ |
| Plane frame: $z_p$ | $3.0 \times 10^{-16}$ |
| Plane frame: $x_p^2/(1/4) + y_p^2/(5/4) - 1$ | $1.8 \times 10^{-15}$ |

Table 3: Residuals of 300 tf samples against the model (`results/logs/check_tf_summary.txt`). Measured ranges: $x_p \in [-0.5000, 0.4999]$, $y_p \in [-1.1180, 1.1180]$.

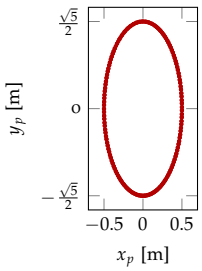



Figure 7: Relative position of AV2 in the plane frame: tf samples (dots) and the ellipse of eq. (2) (grey).

> EVIDENCE Listing 6 (implementation); Figure 6 (three trails in both fixed frames); Table 3 and Figure 7 and `tf_samples.csv` (numerical check).

## 5 *Mathematical Derivations* [10 pts]

*Notation.* $T_b^a \in SE(3)$ is the pose of frame $b$ in frame $a$; it maps homogeneous coordinates as $\tilde{\boldsymbol{p}}^a = T_b^a \tilde{\boldsymbol{p}}^b$, and $T_c^a = T_b^a T_c^b$. $\boldsymbol{o}_b^a$ is the origin of frame $b$ expressed in frame $a$. Throughout, $c = \cos t$, $s = \sin t$, and $R_z(t) = \left[\begin{smallmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]$. From the problem statement,

$$T_1^w = \begin{bmatrix} c & -s & 0 & c \\ s & c & 0 & s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad T_2^w = \begin{bmatrix} 1 & 0 & 0 & s \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

*1. AV2 moves on an arc of a parabola.* $\boldsymbol{o}_2^w = (s, 0, \cos 2t)^T$. With $\cos 2t = 1 - 2\sin^2 t$ the coordinates satisfy

$$y = 0, \qquad z = 1 - 2x^2, \qquad x = \sin t \in [-1, 1].$$

The path is therefore the part $-1 \le x \le 1$ of the parabola $z = 1 - 2x^2$ in the $x$–$z$ plane, with vertex $(0, 0, 1)$ and end points $(\pm 1, 0, -1)$; AV2 sweeps it back and forth with period $2\pi$.

*2. Position of AV2 in AV1's frame.* With $T_1^w = \left[\begin{smallmatrix} R & \boldsymbol{o}_1^w \\ \mathbf{0}^T & 1 \end{smallmatrix}\right]$, $R = R_z(t)$, the inverse is $(T_1^w)^{-1} = \left[\begin{smallmatrix} R^T & -R^T \boldsymbol{o}_1^w \\ \mathbf{0}^T & 1 \end{smallmatrix}\right]$, and $R^T \boldsymbol{o}_1^w = (c^2 + s^2,\ -sc + cs,\ 0)^T = (1, 0, 0)^T$. Hence

$$(T_1^w)^{-1} = \begin{bmatrix} c & s & 0 & -1 \\ -s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

$$T_2^1 = (T_1^w)^{-1} T_2^w = \begin{bmatrix} c & s & 0 & sc - 1 \\ -s & c & 0 & -s^2 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where the last column is $R^T \boldsymbol{o}_2^w - R^T \boldsymbol{o}_1^w = (cs, -s^2, \cos 2t)^T - (1, 0, 0)^T$. Therefore

$$\begin{aligned} \boldsymbol{o}_2^1(t) &= \left(\sin t \cos t - 1,\ -\sin^2 t,\ \cos 2t\right)^T \\ &= \left(\tfrac{1}{2}\sin 2t - 1,\ \tfrac{1}{2}\cos 2t - \tfrac{1}{2},\ \cos 2t\right)^T. \end{aligned} \tag{1}$$

Simply subtracting the world positions, $\boldsymbol{o}_2^w - \boldsymbol{o}_1^w$, would give the offset in world axes; the rotation $R^T$ is needed to express it in AV1's axes. $\boldsymbol{o}_2^1$ has period $\pi$, whereas the full relative pose $T_2^1$ has period $2\pi$ because of its rotation block. Check values: $\boldsymbol{o}_2^1(0) = (-1, 0, 1)$, $\boldsymbol{o}_2^1(\pi/4) = (-\tfrac{1}{2}, -\tfrac{1}{2}, 0)$, $\boldsymbol{o}_2^1(\pi/2) = (-1, -1, -1)$.

*3. The relative path is planar.* From eq. (1), $y = -s^2$ and $z = 1 - 2s^2 = 1 + 2y$ for every $t$, so all points lie in the plane

$$\Pi:\ z - 2y - 1 = 0 \quad \text{(in AV1's frame)}, \qquad \boldsymbol{n} = \tfrac{1}{\sqrt{5}}(0, -2, 1)^T.$$

*4. Coordinates in the plane.* Take the origin $\boldsymbol{p}^1 = (-1, -\tfrac{1}{2}, 0)^T$, which lies on $\Pi$ ($0 + 1 - 1 = 0$), and the orthonormal axes

$$\boldsymbol{x}_p = (1, 0, 0)^T, \qquad \boldsymbol{y}_p = \tfrac{1}{\sqrt{5}}(0, 1, 2)^T,$$
$$\boldsymbol{z}_p = \boldsymbol{x}_p \times \boldsymbol{y}_p = \tfrac{1}{\sqrt{5}}(0, -2, 1)^T = \boldsymbol{n}.$$

$\boldsymbol{x}_p$ and $\boldsymbol{y}_p$ are orthogonal to $\boldsymbol{n}$ and so lie in $\Pi$. With $R_p^1 = [\boldsymbol{x}_p\ \boldsymbol{y}_p\ \boldsymbol{z}_p]$,

$$T_p^1 = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & \frac{1}{\sqrt{5}} & -\frac{2}{\sqrt{5}} & -\frac{1}{2} \\ 0 & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$
$$(T_p^1)^{-1} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} & \frac{1}{2\sqrt{5}} \\ 0 & -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} & -\frac{1}{\sqrt{5}} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Using $\boldsymbol{o}_2^1 - \boldsymbol{p}^1 = (\tfrac{1}{2}\sin 2t,\ \tfrac{1}{2}\cos 2t,\ \cos 2t)^T$, $\tilde{\boldsymbol{o}}_2^p = (T_p^1)^{-1}\,\tilde{\boldsymbol{o}}_2^1$ gives

$$x_p = \tfrac{1}{2}\sin 2t,$$
$$y_p = \tfrac{1}{\sqrt{5}}\big(\tfrac{1}{2}\cos 2t + 2\cos 2t\big) = \tfrac{\sqrt{5}}{2}\cos 2t,$$
$$z_p = \tfrac{1}{\sqrt{5}}\big(-\cos 2t + \cos 2t\big) = 0.$$

$z_p$ vanishes as expected, and replacing $t$ by $-t$ or by $\pi/2 - t$ mirrors the curve in the $y_p$ or the $x_p$ axis, so both axes are axes of symmetry.

*5. The path is an ellipse.* Eliminating $t$,

$$\frac{x_p^2}{(1/2)^2} + \frac{y_p^2}{(\sqrt{5}/2)^2} = \sin^2 2t + \cos^2 2t = 1, \tag{2}$$

an axis-aligned ellipse centred at $\boldsymbol{p}^1$ with semi-axes $a = \tfrac{1}{2}$ along $\boldsymbol{x}_p$ and $b = \tfrac{\sqrt{5}}{2} \approx 1.118$ along $\boldsymbol{y}_p$, traversed once every $\pi$ seconds. The tf samples reach $|x_p| = 0.5000$ and $|y_p| = 1.1180$ (Table 3).

> EVIDENCE Homogeneous transforms $T_1^w$, $T_2^w$, $(T_1^w)^{-1}$, $T_2^1$, $T_p^1$, $(T_p^1)^{-1}$ with all intermediate steps; results confirmed against tf in Table 3 and Figure 7.

## 6 *Quaternion Properties* [10 pts]

*Conventions.* A quaternion is $q = (q_1, q_2, q_3, q_4)^T = (\boldsymbol{v}, w)$ with vector part $\boldsymbol{v} = (q_1, q_2, q_3)^T$ and scalar part $w = q_4$, and $[\boldsymbol{v}]_\times$ is the

skew-symmetric matrix with $[v]_\times a = v \times a$. The matrices of the handout[5] are, in block form,

$$\Omega_1(q) = \begin{bmatrix} wI + [v]_\times & v \\ -v^T & w \end{bmatrix}, \qquad \Omega_2(q) = \begin{bmatrix} wI - [v]_\times & v \\ -v^T & w \end{bmatrix}.$$

Applied to $y = (v_y, w_y)$ and $x = (v_x, w_x)$ they give $\Omega_1(x)\, y = \Omega_2(y)\, x = (w_x v_y + w_y v_x + v_x \times v_y,\ w_x w_y - v_x^T v_y)$, which is the quaternion product $x \otimes y$ for *all* $x, y \in \mathbb{R}^4$. Two identities are used below: $[v]_\times^T = -[v]_\times$ and $[v]_\times^2 = vv^T - \|v\|^2 I$.

[5] MIT SPARK Lab. VNAV 2023 Lab 2: ROS Exercises. https://vnav.mit.edu/labs_2023/lab2/exercises.html, 2023a

*1. Orthogonality.* Write $\Omega_i(q) = \left[\begin{smallmatrix} A & v \\ -v^T & w \end{smallmatrix}\right]$ with $A = wI + \epsilon[v]_\times$, $\epsilon = +1$ for $\Omega_1$ and $-1$ for $\Omega_2$. Then

$$\Omega_i^T \Omega_i = \begin{bmatrix} A^T A + vv^T & A^T v - wv \\ v^T A - wv^T & v^T v + w^2 \end{bmatrix}.$$

Since $A^T = wI - \epsilon[v]_\times$ and $\epsilon^2 = 1$, $A^T A = w^2 I - [v]_\times^2 = (w^2 + \|v\|^2) I - vv^T$, so the top-left block is $\|q\|^2 I$. The off-diagonal blocks vanish because $A^T v - wv = -\epsilon\, v \times v = \mathbf{0}$, and the bottom-right entry is $\|q\|^2$. Hence $\Omega_i(q)^T \Omega_i(q) = \|q\|^2 I_4 = I_4$ for a unit quaternion; a square matrix whose transpose is a left inverse also has it as a right inverse, so $\Omega_i(q)\Omega_i(q)^T = I_4$ as well.

The slides print $\Omega_1$ with $(1,3)$ entry $-q_2$ and $(2,3)$ entry $+q_1$; the handout has $+q_2$ and $-q_1$. The slide version is not orthogonal (a random unit $q$ gives $\max |\Omega_1^T \Omega_1 - I| = 0.999$) and is taken as a typo. `scripts/quaternion_check.py` also checks all three properties numerically; this is a sanity check, not a proof.

*Intuition.* For unit $q$ and every unit $x$, $q \otimes x$ and $x \otimes q$ are again unit quaternions, i.e. $\|\Omega_1(q)x\| = \|\Omega_2(q)x\| = 1$. By linearity both maps preserve the length of every vector in $\mathbb{R}^4$, and a linear map that preserves lengths is orthogonal.

*2. Mapping $q$ to the identity.* The last column of both matrices is $q$, so $\Omega_i(q)\, e_4 = q$ with $e_4 = (0,0,0,1)^T$, the identity rotation ($q \otimes 1 = 1 \otimes q = q$). Multiplying by $\Omega_i(q)^T$ and using part 1,

$$\Omega_i(q)^T q = \Omega_i(q)^T \Omega_i(q)\, e_4 = e_4 = (0,0,0,1)^T.$$

Equivalently, $\Omega_1(q)^T = \Omega_1(q^*)$ with the conjugate $q^* = (-v, w)$, so the statement reads $q^* \otimes q = 1$.

*3. Commutation.* For arbitrary $x, y, z \in \mathbb{R}^4$, using $\Omega_1(a)b = a \otimes b$, $\Omega_2(b)a = a \otimes b$ and the associativity of quaternion multiplication,

$$\Omega_1(x)\Omega_2(y)\, z = x \otimes (z \otimes y) = (x \otimes z) \otimes y = \Omega_2(y)\Omega_1(x)\, z.$$

Since this holds for every $z$, $\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$. For the second identity note that $\Omega_2(y)^T = \left[\begin{smallmatrix} w_y I + [v_y]_\times & -v_y \\ v_y^T & w_y \end{smallmatrix}\right] = \Omega_2(y^*)$, because $[-v_y]_\times = -[v_y]_\times$. Applying the first identity to $x$ and $y^*$ gives $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T \Omega_1(x)$. Neither argument needs $x$ or $y$ to be unit quaternions.

> EVIDENCE Proofs of the three properties with block matrices; note on the sign difference in the slides; numerical sanity check in `results/logs/quaternion_check.txt` (largest deviation

> $1.8 \times 10^{-15}$ over 1000 random cases).

## 7 Discussion

*Problems met.*

- *Expired ROS key.* Older guides import the ROS key with `apt-key`; that key expired in June 2025. The `ros-apt-source` package installs the current key and source.
- *Network in WSL.* Windows uses a local proxy, which WSL 2 cannot reach in its default NAT mode; `raw.githubusercontent.com`, needed by `rosdep`, timed out. Switching WSL to mirrored networking with `autoProxy` passes the Windows proxy through.
- *Invisible windows.* When WSL was started from the SSH session, WSLg created its display client in that non-interactive Windows session and the turtlesim window stayed unmapped. Starting WSL from the desktop first fixed it.
- *Display manager.* `desktop-full` pulls in `gdm3`, which tried to start under systemd inside WSL; it was disabled.
- *Missing meshes.* RViz drew the grid and TF axes but not the drone meshes with the WSLg D3D12 OpenGL driver (OpenGL 3.1 on an RTX 4070 SUPER). Software rendering (`LIBGL_ALWAYS_SOFTWARE=1`) draws them correctly at 30 fps, but RViz then uses about 1.7 CPU cores. When the plotting timer is delayed, consecutive trail samples are far apart and the line strip shows a short chord; it leaves the trail once the gap drops out of the buffer.
- *Template and slides.* The member-order defect in `plots_publisher_node` and the sign typo in the slides' $\Omega_1$ were found and handled as described above.

*Reflections.* The lab turns abstract transform notation into things that can be seen. Three details decided whether the result was right: the direction of a transform (`lookupTransform(ref, dest)` gives $T^{\text{ref}}_{\text{dest}}$, and the swapped call draws a different curve), the orientation of AV1 (with yaw $t + \pi/2$ the relative path turns by 90° about $z_1$ and lies on $z = 1 + 2x$ instead, so the plane and the hinted origin no longer fit), and the need to rotate, not only subtract, when changing frames. Checking the ROS output against the closed-form result made each of these testable instead of a matter of how the RViz picture looks.

## 8 Conclusions

ROS Noetic was installed and tested with turtlesim. The two-drone scenario was analysed through its nodes, topics and launch file, the

drone poses were published with `tf2`, and the relative transform was looked up to draw the three trails. The relative path of AV2 in AV1's frame is an ellipse on the plane $z - 2y = 1$ with semi-axes $\frac{1}{2}$ and $\frac{\sqrt{5}}{2}$, derived with homogeneous transforms and matched by the tf data to round-off level. The quaternion multiplication matrices were shown to be orthogonal, to map $q$ to the identity, and to commute.

## *References*

MIT SPARK Lab. VNAV 2023 Lab 2: ROS Exercises. https://vnav.mit.edu/labs_2023/lab2/exercises.html, 2023a.

MIT SPARK Lab. VNAV-labs, release VNAV2023 (commit 7d234d8). https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2023, 2023b.

Open Robotics. ROS signing key migration guide. https://discourse.openrobotics.org/t/ros-signing-key-migration-guide/43937, 2025.